CSE470: Software Engineering

Software Requirements Specification (SRS)

End-of-Semester Submission Template

"Voyenta"

International Tour Planner

Your Journey, Your Way.

Prepared By

Student ID	Name
23201054	Tahmid Islam
23201162	Sumaiya Islam Aritra
24141229	Afnan Mazumder
24101084	Ramisa Faria

Project Information

Section	11
Section Faculty	Nazmus Sakib Touhid
GitHub Repository Link	
Tech Stack Used	React, Fast API, PostgreSQL, Laravel

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements and expected functionality of **Voyenta**, an international tour planning and management web application. The application aims to help independent travelers, groups, and families organize their trips in one place by providing destination discovery, personalized recommendations, itinerary generation, budget planning, expense tracking, travel information, and trip management tools.

1.2 Scope

Voyenta is a responsive web application designed to simplify international travel planning and management. It allows independent travelers, groups, and families to explore destinations, create personalized day-by-day itineraries, check visa and entry requirements, estimate and track expenses in multiple currencies, and compare flight and accommodation options using data maintained within the platform.

The application also provides interactive maps and route planning, weather information, smart packing lists, local transport and safety information, language assistance, group trip collaboration, travel document storage, notifications and reminders, and itinerary sharing and export. Administrators can manage destination and visa information, maintain platform data and view usage analytics.

In scope: User registration and profiles, travel preferences, destination discovery, itinerary generation, personalized recommendations, visa information, budget planning, currency conversion, expense tracking, flight and accommodation search, maps and routes, weather forecasts, packing assistance, safety information, language support, group collaboration, travel document management, notifications, trip sharing, PDF and calendar exports, reviews, and an administrative dashboard.

Out of scope:

- Real ticket issuance, real hotel reservations with airlines and hotels. Flight and accommodation results come from data maintained in the platform, and bookings are records the traveler keeps for organization.
- Submitting visa applications directly to government authorities. Visa information is provided for guidance, with links to official sources and a reminder to verify current requirements.
- Developing native Android or iOS applications. The project focuses on a responsive web application.
- Providing live GPS tracking, offline turn-by-turn navigation, or emergency dispatch services.

2. Overall Description

2.1 Product Perspective

The International Tour Planner is a standalone web product. Travelers today spread one trip across many tools: search sites for flights and hotels, note apps for itineraries, spreadsheets for budgets, separate currency converters, and messaging groups for coordination. This application brings those tasks into one place around a single trip record. It uses public data services (maps, routing, weather, exchange rates, and country and attraction data) through their APIs while keeping trips, bookings, budgets, and documents in its own database.

2.2 Product High-Level Summary

- **Trip management:** Travelers can create, view, update, and manage trips, including destinations, travel dates, and group members.
- **Personalization:** Users can save their travel interests and receive destination and attraction recommendations based on matching categories and preferences.
- **Destination discovery:** Users can search for countries, cities, and attractions and explore destination information using filters and detail pages.
- **Itinerary planning:** Users can create day-by-day itineraries, organize activities, and receive automatically generated suggestions based on trip dates, interests, and attraction locations.
- **Visa and entry information:** Users can check available visa and entry requirement information based on passport nationality and destination, with links to official sources.
- **Budget and expense management:** Users can estimate trip costs, record expenses in different currencies, compare planned and actual spending, and calculate shared expenses.
- **Flight and accommodation management:** Users can search and compare flight and accommodation listings maintained within the platform and save booking details for reference.
- **Maps and weather:** Users can view destinations and attractions on interactive maps, check available weather forecasts, and access route information.
- **Packing assistance:** The system generates packing suggestions based on destination, trip duration, weather, and planned activities.
- **Safety and language assistance:** Users can access destination-specific emergency information, local transport guidance, useful phrases, and cultural tips.
- **Group trip planning:** Trip owners can invite other users to view or edit shared itineraries according to their assigned permissions.
- **Travel document management:** Users can upload and organize trip-related documents with access restricted to authorized users.
- **Notifications and reminders:** The system provides reminders for important trip dates, saved booking deadlines, and document expiry dates.
- **Trip sharing and reviews:** Users can share read-only trip plans, export itineraries as PDF files, and record reviews of visited destinations and attractions.
- **Administration and analytics:** Administrators can manage destination, attraction, visa, and listing data, handle reported content, and view basic platform usage statistics.

2.3 Users and Characteristics

User Role	Description
Traveler	The main user who creates trips, explores destinations, manages itineraries and budgets, records expenses, stores travel documents, and uses travel-planning tools.
Group Member	A registered user invited to a shared trip. Depending on assigned permissions, the member can view or edit the itinerary and participate in group planning.
Administrator	Manages destination and attraction information, visa data, flight and accommodation listings, user accounts, and reported content. The administrator can also access basic platform analytics.

2.4 Operating Environment

- A responsive web application used in current desktop and mobile browsers (Chrome, Edge, Firefox, Safari).
- The server runs PHP 8.3+ with Laravel, PostgreSQL, and a separate Python 3 FastAPI service for AI features, deployable on Linux hosting or containers.
- An internet connection is needed for external APIs (maps, routing, weather, exchange rates, attraction data).

2.5 Design and Implementation Constraints

This project follows the constraints defined in the CSE470 Project Outline, including mandatory MVC architecture and the approved tech-stack policy. State your specific choices below.

(a) Primary tech stack: Laravel (PHP) backend exposing a REST API, a React single-page frontend, PostgreSQL through Laravel's Eloquent ORM.

(b) MVC framework and structure: Laravel provides the Model and Controller layers, and React provides the View layer.

- **Model**: Eloquent models in app/Models (for example Trip, ItineraryItem, Expense, Booking) hold data and data rules. Larger business rules live in service classes in app/Services (for example BudgetService, PackingRuleService) that controllers call.
- **View**: React pages and components in the frontend (for example TripPage, ItineraryBoard, BudgetDashboard) handle presentation and user interaction only and get data through the REST API
- **Controller**: Laravel controllers in app/Http/Controllers (for example TripController, ExpenseController) receive requests, validate them with Form Request classes, call models and services, and return JSON to the React views.

(c): AI-backed features: the Personalized Recommendation Engine and the attraction clustering step of the Smart Itinerary Generator run in a separate Python FastAPI service using scikit-learn. Laravel calls this service through a defined REST interface, sends trip and preference data, and receives ranked recommendations or day clusters. All other logic stays in the Laravel application.

(d) Faculty-approved exception: the Python AI service in (c) requires approval from the section faculty under Section 5 of the Project Outline. Approval status: pending.

2.6 Assumptions and Dependencies (Optional)

- The free tiers of the external APIs (OpenStreetMap tiles, OpenRouteService, Open-Meteo, Frankfurter, countries.dev, OpenTripMap, Wikivoyage, Overpass and Resend) remain available within their rate limits. Responses are cached so the application stays within those limits.
- Flight data and accommodation listings are entered or imported by admins and travel agents. There is no live airline or hotel booking integration.
- Visa rules come from a country-pair dataset maintained by admins. Official rules can change before the dataset is updated, so every result shows a last-updated date, a disclaimer and a link to the official source. Exchange rates are daily reference rates, not live trading rates.
- Users have an internet connection and a current desktop or mobile browser.
- The FastAPI AI service runs alongside the Laravel application. If it is unavailable, recommendations fall back to popular places and the itinerary generator falls back to grouping attractions by distance.
- The project is hosted on a server that supports PHP, PostgreSQL and Python.

3. System Architecture

3.1 Architectural Overview

React components never query the database or external services directly. Every user action becomes a REST call to a Laravel controller, which validates the input, calls service classes and Eloquent models, and returns JSON for the view to render. Services are the only part that talks to the FastAPI AI service and to external APIs, so API keys stay on the server.

Example request trace (logging an expense):

1. **View:** The traveler submits the form on ExpensePage (React), which sends `POST /api/trips/{trip}/expenses`.
2. **Controller:** `ExpenseController@store` validates the request with `StoreExpenseRequest` and checks that the user belongs to the trip.
3. **Model/Service:** `CurrencyService` gets the cached exchange rate, and the `Expense` model saves the original amount, currency, rate used, and converted amount.
4. **View:** The controller returns the saved expense as JSON, and `BudgetDashboard` updates the planned vs actual totals.

3.2 Technology Stack

Layer	Technology	Purpose
Frontend (View)	React (Vite), React Router, Axios, Tailwind CSS, React-Leaflet, Recharts, Laravel Echo client	Pages and components, API calls, maps, charts and real-time updates
Backend (Controller and Model)	Laravel (PHP 8.3+), Eloquent ORM, Form Requests	REST API, request validation, business logic and data access
Authentication	Laravel Sanctum, Laravel Socialite (Google login)	SPA token authentication and Google sign-in
Real-time	Laravel Reverb (WebSockets)	Live updates for group trip collaboration
Background jobs	Laravel Scheduler and Queues	Reminders, exchange-rate refresh, price alerts
Documents and export	barryvdh/laravel-dompdf, generated .ics files	Itinerary PDF and calendar export
Database	PostgreSQL	Relational storage for all entities

AI/ML service	Python 3, FastAPI, scikit-learn	Recommendation engine and itinerary attraction clustering
Version control	Git and GitHub	Source control, branches, pull requests, sprint evidence

4. Requirements and Features

Note: Registration, login, Google sign-in, email verification, password reset, and profile editing are supporting functions and are not counted as features, following Section 3 of the Project Outline. The Sprint Completed and Status columns are filled in as sprints finish.

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R1	Travelers can create and manage their trips	**Trip Management:** Create, view, edit, duplicate, and archive trips with title, dates, ordered destination stops, and travelers. The trip dashboard shows each trip's status (planning, upcoming, ongoing, completed).	Traveler	**Model:** Trip, TripDestination **View:** TripListPage, TripFormPage		

				Controller: TripController		
R2	Traveler Preferences	**Traveler Preference and Interest Engine:** Onboarding quiz for interests, pace, and budget band. Each interest has a weight that rises when the traveler saves a suggestion and falls when they reject one.	Traveler	**Model:** Profile, InterestWeight **View:** OnboardingQuiz, PreferencesPage **Controller:** PreferenceController		

R3	The system personalizes suggestions to each traveler	**Personalized Recommendation Engine:** Suggests attractions, restaurants, and destinations by matching place tags to interest weights (cosine similarity) and by collaborative filtering from similar travelers, computed in the AI service. Each suggestion shows a reason; new users see popular places.	Traveler	**Model:** Attraction, InterestWeight, Review **View:** Recommendations Panel **Controller:** RecommendationController (calls the FastAPI service)		

R4	Travelers can discover destinations	**Destination Search and Explorer:** Search by country, city, region, or theme with filters for budget level, best month, climate, trip length, and safety. Destination pages show overview, attractions, cuisine, cost index, best season, photos, and a map, using cached data from countries.dev, OpenTripMap, and Wikivoyage.	Traveler	**Model:** Destination, Attraction **View:** ExplorePage, DestinationPage **Controller:** DestinationController		

R5	The system builds a day-by-day itinerary	**Smart Itinerary Generator:** Takes destinations, dates, budget, interests, travelers, and pace. Groups nearby attractions into days (k-means in the AI service), orders each day to shorten travel using OpenRouteService distances, and respects opening hours. Produces morning, afternoon, and evening blocks; one day or the whole trip can be regenerated or edited by hand.	Traveler	**Model:** ItineraryDay, ItineraryItem, Attraction **View:** ItineraryBoard **Controller:** ItineraryController (calls the FastAPI service)		

R5	Travelers can check entry requirements	**Visa and Entry Requirement Checker:** For a passport nationality and destination, shows visa-free, visa on arrival, e-visa, or embassy visa, allowed stay, required documents, passport validity rules, and transit warnings, with official links, a last-updated date, and a disclaimer. Warns on the trip page when a stop needs a visa.	Traveler, Admin	**Model:** VisaRequirement **View:** VisaCheckerPage **Controller:** VisaController		

R6	Travelers can plan and control their trip budget	**Budget Planner and Cost Estimator:** Estimates cost per category (flights, stay, food, transport, activities, visa, insurance, shopping, emergency buffer) from cost index, trip length, travelers, and comfort level. Shows planned vs actual spending with progress bars and over-budget alerts, and suggests cheaper swaps using a knapsack optimizer.	Traveler	**Model:** Budget, Expense, BudgetService **View:** BudgetDashboard **Controller:** BudgetController		

R7	Travelers can plan and control their trip budget	**Multi-Currency Converter and Expense Tracker:** Daily exchange rates from Frankfurter, cached and refreshed on a schedule. Expenses are logged in local currency and converted to home currency, storing the rate used. Costs can be split among group members with a who-owes-whom summary and CSV or PDF export.	Traveler, Group Member	**Model:** Expense, ExpenseShare, ExchangeRate **View:** ExpensePage **Controller:** ExpenseController		

R8	Travelers can compare flights and organize stays	**Flight Search and Comparison:** Searches admin-managed flight data by origin, destination, dates, passengers, and cabin class. Sorts and filters by price, duration, stops, and departure time, supports multi-city trips, shows a flexible-date price calendar, and sends price-drop alerts. Admins maintain the flight data.	Traveler, Admin	**Model:** Flight, PriceAlert **View:** FlightSearchPage, FlightAdminPage **Controller:** FlightController		

R9	Travelers can compare flights and organize stays	**Accommodation Finder and Booking Management:** Filters stays by price, rating, distance, amenities, and property type, with map search and price pins. Saves booking records with confirmation number, dates, status (planned, confirmed, cancelled), and cancellation-deadline reminders.	Traveler, Travel Agent	**Model:** Accommodation, Booking **View:** StaySearchPage, BookingsPage **Controller:** AccommodationController, BookingController		
R10	Travelers can plan movement, weather and packing for the trip	**Interactive Map and Route Planner:** Shows hotels, attractions, and restaurants on the trip map, colored by day, with routes and travel times for walking, cycling, or driving from OpenRouteService. Travelers can save places to the trip.	Traveler	**Model:** ItineraryItem, Attraction, RouteService		

				View: TripMapPage **Controller:** RouteController		
R11	Travelers can plan movement, weather and packing for the trip	**Weather Forecast and Best-Time Advisor:** Shows daily forecasts for itinerary days and climate averages for trips further ahead (Open-Meteo), advises the best months to visit, flags bad-weather days, and suggests indoor alternatives.	Traveler	**Model:** WeatherService, ItineraryDay **View:** WeatherPanel **Controller:** WeatherController		

R12	Travelers can plan movement, weather and packing for the trip	**Smart Packing List Generator:** Generates a checklist from weather, trip length, activities, and destination rules (for example plug type), grouped into documents, clothing, electronics, toiletries, and medicine, with check-off progress and sharing.	Traveler	**Model:** PackingItem, PackingRule, PackingRuleService **View:** PackingListPage **Controller:** PackingController		
R13	Travelers can stay safe and communicate abroad	**Local Transport, Safety and Emergency Information:** Shows emergency numbers, embassy contacts, advisory level, and common scams; nearby hospitals and police on the map (Overpass); a local transport guide with approximate fares; and a printable emergency card.	Traveler, Admin	**Model:** SafetyInfo **View:** SafetyPage		

				Controller: SafetyController		
R14	Travelers can stay safe and communicate abroad	**Language Helper and Cultural Tips:** Phrasebook for the top 10 languages (greetings, food, directions, emergencies) with pronunciation and saved favorites, plus etiquette notes on tipping, dress code, customs, and business hours. Machine translation is added later.	Traveler	**Model:** Phrase, CulturalTip **View:** LanguageHelperPage **Controller:** PhraseController		

R15	Groups can plan one trip together	**Group Trip Collaboration:** Invite members by email or link as owner, editor, or viewer. The shared itinerary updates in real time (Laravel Reverb), members vote and comment on items, and an activity history shows who changed what.	Traveler, Group Member	**Model:** TripMember, Comment, Vote, ActivityLog **View:** CollaboratorsPanel, ItineraryBoard **Controller:** TripMemberController, CommentController		

R16	Travelers can keep travel documents secure	**Travel Document Vault:** Upload passport copies, visas, tickets, vouchers, and insurance, linked to a trip and traveler. Files are encrypted at rest (AES), document numbers are masked, expiry dates trigger reminders, and a quick-access view supports airport check-in.	Traveler	**Model:** Document **View:** DocumentVaultPage **Controller:** DocumentController		

R17	The system reminds travelers of upcoming trip events	**Notifications, Reminders and Trip Timeline:** Scheduled email and in-app alerts for check-in opening, flights within 24 hours, visa deadlines, payments, and document expiry; a daily digest on travel days; one timeline of flights, stays, and activities; and notification preferences.	Traveler	**Model:** Notification **View:** TimelinePage, NotificationSettings **Controller:** NotificationController		

R18	Travelers can share, export and review trips	**Trip Sharing, PDF/Calendar Export and Reviews:** Exports the itinerary as PDF and .ics, creates a public read-only share link with privacy controls, and records an after-trip journal with photos, ratings, and reviews that feed the recommendation engine.	Traveler	**Model:** Review, JournalEntry, Trip **View:** SharePage, JournalPage **Controller:** ExportController, ReviewController		

R19	Administrators manage the platform and view usage	**Admin Dashboard and Analytics:** Manage destinations, attractions, visa data, flights, users, and reported content. Charts show popular destinations, average budget per country, active users, and seasonal trends; stale visa or price data is flagged and admin actions are logged.	Admin	**Model:** AuditLog and reference models **View:** AdminDashboard **Controller:** admin controllers		

4.1 Feature Count Summary

Minimum features required (per Project Outline, Section 7)	20
Total features listed in Section 4 above	20
Total features fully completed (“Done” per Section 7.2 of the Project Outline)	

5. External Interface Requirements (Optional)

Interface Type	Description
User Interface	Responsive React web interface for desktop and mobile browsers: trip dashboard, destination pages, itinerary board, budget dashboard, maps, forms with inline validation messages, and an admin dashboard.
Hardware Interface	No special hardware. Runs on any computer or phone with a current browser. Uses the device location (with permission) for nearby places and the standard file picker or camera for document uploads.
Software Interface (APIs, third-party services)	Leaflet with OpenStreetMap tiles (maps); OpenRouteService (routes and distance matrix); Open-Meteo (forecast and climate data); Frankfurter (exchange rates); countries.dev (country data); OpenTripMap and Wikivoyage (attractions and travel guides); Overpass (nearby hospitals, police and hotels); Resend (email); Google OAuth (sign-in); Python FastAPI AI service (recommendations and clustering); PostgreSQL (database).
Communication Interface	HTTPS for all traffic. React calls the Laravel REST API with JSON and Sanctum tokens. Laravel calls the FastAPI service over REST with JSON. Laravel Reverb WebSockets push live updates for group trips. Email is sent through the Resend API.

6. Non-Functional Requirements (Optional)

7. Sprint History and Review Ceremony Log

Record the goals and attainment shared during each weekly sprint review ceremony, per Section 7.1 of the Project Outline.

Sprint	Goals Set (Start of Week)	Attainment Reported (End of Week)	Features Completed This Sprint
Sprint 1			
Sprint 2			
Sprint 3			
Sprint 4			

7.1 Deviations and Lessons Learned

[Briefly explain any sprint where attainment did not match the planned goals, why it happened, and what was adjusted for the following sprint.]

8. Testing Summary

[Briefly describe how each major feature was tested (manual testing steps, sample test cases, or automated tests). A short table or list is acceptable.]

9. AI Tool Usage Disclosure

Per the Project Outline's Academic Integrity & AI Usage Policy (Section 6), disclose every instance where a generative AI tool was used during development. Leave the table empty (do not delete it) if no AI tool was used.

Feature / Task	AI Tool Used	How It Was Used (be specific)	I Can Fully Explain This Code (Yes/No)

[By submitting this document, you confirm that you are able to personally explain, modify, and debug every piece of code listed above during your viva.]

10. Individual Contribution Summary

11. Pre-Submission Checklist

Checklist Item	Confirmed (Yes/No)
The application implements the MVC architecture	
At least 20 features are listed and each is correctly counted per the granularity rule	
No forbidden framework (e.g. Django, Flask as primary backend) was used	
All four sprints' goals and attainment are recorded in Section 7	
The GitHub repository is up to date and the section faculty is added as a collaborator	
The AI Usage Disclosure table (Section 10) is complete and honest	
I can personally explain and defend every feature listed in this document during the viva	